РЕКЛАМА
При поддержке Хабр Курсы РЕКЛАМА
Хабр Курсы
Взлетайте в профессии

512K+
Охват за 30 дней
OTUS
604,6 Рейтинг
187 303 Подписчики
Подписаться

Andrey_Biryukov
1 час назад

Сделаем Python безопасным... снова

Средний 33 мин 3.1K

Блог компании OTUS, Python*, Информационная безопасность*, Программирование*

Туториал

Все мы любим Python за то, что он дает нам свободу: динамическую типизацию, кроссплатформенность, огромное количество библиотек и многое другое. Но зачастую эта свобода становится кошмаром для security-инженеров и архитекторов, когда речь заходит о высоконагруженных системах с серьезными требованиями к безопасности. В этой статье мы поговорим о том, как перехватить выполнение Python-кода, запретить опасные вызовы и построить систему контрактов без изменения исходников.

Плохая динамика

Но сначала давайте немного «поучимся плохому» и рассмотрим несколько примеров, демонстрирующих проблемы динамического выполнения:

Начнем с наиболее известного примера — выполнения произвольного кода.

```
user_input = "import os; os.system('rm -rf /')"
eval(user_input)
```

Объяснить с SourceCraft

> Как видно, здесь `eval()` выполняет содержимое `user_input` без проверки. Никогда не делайте так!

Далее, рассмотрим подмену методов в рантайме.

```
import requests
requests.get = lambda args, *kwargs: print("Перехвачено!")
```

Объяснить с

Здесь мы подменяем стандартный метод `requests.get` на собственную lambda-функцию. Это затрагивает все импорты `requests` в нашем приложении и любых сторонних библиотеках, которые его используют. Другие части программы, ожидающие настоящий ответ от сервера, внезапно начнут получать `None` (потому что `print` возвращает `None`).

При поддержке Хабр Курсы РЕКЛАМА

к файловой системе.

```
def read_sensitive_data():
    with open('/etc/passwd', 'r') as f:
        return f.read()
```

Объяснить с

В домашних скриптах так делать может и можно, но в продуктиве точно не стоит.

Что мы можем предложить для борьбы с этим? Первое, что приходит в голову это старый, добрый статический анализ. Однако, от таких ошибок данный инструмент не очень защитит. Дело в том, что он проверяет типы, но не контролирует, что произойдёт во время выполнения. И для решения этой проблемы нужны более глубокие механизмы.

Для примера давайте рассмотрим несколько стандартных инструментов и их недостатки. Так, mypy проверяет типы аргументов и возвращаемые значения. При этом утилита не проверяет безопасность вызовов и возможные побочные эффекты. Инструмент **pylint** проверяет стиль кода и потенциальные баги. В свою очередь, сканер **bandit** ищет известные уязвимости (`hardcoded secrets` , `eval`).

Хочу свой анализатор

Но что делать, если нам не хватает тех возможностей, которые предлагают готовые инструменты? В таком случае мы можем разработать собственный анализатор, например, на основе модуля `ast` .

Этот модуль позволяет анализировать структуру кода без его выполнения. Вот небольшой пример.

```
import ast
import sys

class SecurityAnalyzer(ast.NodeVisitor):

    """Анализатор безопасности, который запрещает опасные паттерны"""

    def init(self):
        self.violations = []
        self.forbidden_functions = {'eval', 'exec', 'compile', '__import__'}
        self.forbidden_attrs = {'os.system', 'subprocess.Popen', 'os.remove'}

    def visit_Call(self, node):

        # Запрет на вызов eval/exec

        if isinstance(node.func, ast.Name):
            if node.func.id in self.forbidden_functions:
                self.violations.append({
                    'line': node.lineno,
                    'message': f"Запрещён вызов {node.func.id}()",
                    'severity': 'CRITICAL'
                })

        # Запрет на опасные методы
        elif isinstance(node.func, ast.Attribute):
            attr_path = self._get_attr_path(node.func)
```

При поддержке Хабр Курсы РЕКЛАМА

```
                self.violations.append({
                    'line': node.lineno,
                    'message': f"Запрещён вызов {attr_path}",
                    'severity': 'HIGH'
                })

        self.generic_visit(node)

    def visit_Import(self, node):

        # Проверка импортов

        for alias in node.names:
            if alias.name in ['os', 'subprocess', 'socket', 'ctypes']:
                self.violations.append({
                    'line': node.lineno,
                    'message': f"Опасный импорт: {alias.name}",
                    'severity': 'MEDIUM'
                })

    def visit_ImportFrom(self, node):
        self.visit_Import(node)

    def getattr_path(self, attr_node):

        """Рекурсивное восстановление полного имени атрибута (a.b.c)"""

        if isinstance(attr_node, ast.Attribute):
            return f"{self._get_attr_path(attr_node.value)}.{attr_node.attr}"
        elif isinstance(attr_node, ast.Name):
            return attr_node.id
        return ""

    def analyze_file(self, filename):
        with open(filename, 'r') as f:
            tree = ast.parse(f.read(), filename=filename)
        self.visit(tree)
        return self.violations

# Использование

if name == "__main__":
    analyzer = SecurityAnalyzer()
    violations = analyzer.analyze_file("target_script.py")

    for v in violations:
        print(f"[{v['severity']}] Line {v['line']}: {v['message']}")

    if violations:
        sys.exit(1)  # Блокируем выполнение
```

Объяснить с

Давайте посмотрим, что делает этот код. Переданный на анализ файл `target_ script.py` читается и преобразуется в AST — древовидную структуру, где каждый узел представляет элемент кода (функция, вызов, импорт и так далее). Затем выполняется обход дерева с помощью анализатора **SecurityAnalyzer**, который вызывает соответствующие методы (`visit_Call` , `visit_Import` и так далее) для каждого узла.

При поддержке Хабр Курсы РЕКЛАМА

критичности. Так, запрещённым функциям, таким как eval, exec и др., будет присвоен `CRITICAL`, опасным методам (`os.system` и др.) — `HIGH`, а опасному импорту (`os`, `subprocess` и др.) — `MEDIUM`. Все найденные проблемы сохраняются в `self.violations` и печатаются в формате [УРОВЕНЬ] Line X: Сообщение. Если есть нарушения, скрипт завершается с кодом 1 (ошибка).

Так, для приведённого в качестве примера плохого кода фрагмента:

```
import os

eval("print('Hello')")
os.system("rm -rf /")
```

Объяснить с

Мы получим следующий отчет:

> *`[MEDIUM] Line 1`: Опасный импорт: `os`*
>
> *`[CRITICAL] Line 2`: Запрещён вызов `eval()`*
>
> *`[HIGH] Line 3`: Запрещён вызов `os.system`*

Таким образом, мы можем выполнить анализ исходного кода самостоятельно. Однако одного статического анализа недостаточно, так как код может генерироваться динамически, загружаться из плагинов и так далее. Тут нам на помощь приходят перехватчики выполнения. Так функция `settrace` позволяет установить обработчик, который вызывается при каждом событии выполнения.

```
import sys
import dis

from typing import Optional, Any, Dict, Set
from functools import wraps

class ExecutionMonitor:

    """

    Мониторинг выполнения кода с защитой от опасных операций.

    Исправленная версия с корректной работой всех компонентов.

    """

    def init(self):
        self.calls_count: int = 0
        self.max_depth: int = 1000
        self.is_tracing: bool = False
        self.forbidden_functions: Set[str] = {'eval', 'exec', '__import__', 'compile'}
        self.forbidden_modules: Set[str] = {'os', 'subprocess', 'socket', 'sys', 'shut

    def trace_calls(self, frame, event: str, arg: Any) -> Optional[callable]:

        """
```

При поддержке Хабр Курсы РЕКЛАМА

```
        """

        if event == 'call':
            # Защита от рекурсивного зацикливания трассировщика
            if self.is_tracing:
                return self.trace_calls

            self.is_tracing = True

            try:
                # Проверка глубины рекурсии
                self.calls_count += 1
                if self.calls_count > self.max_depth:
                    raise RecursionError(
                        f"Превышена максимальная глубина вызовов ({self.max_depth})"
                    )



                # Проверяем имя вызываемой функции
                func_name = frame.f_code.co_name

                # Проверка опасных функций
                if func_name in self.forbidden_functions:
                    raise RuntimeError(
                        f"Запрещён вызов опасной функции: {func_name}()"
                    )

                # Проверка опасных модулей (если импортируется функция из них)
                if func_name == '__import__':
                    # В контексте импорта можно проверить аргументы

                    local_vars = frame.f_locals
                    if 'name' in local_vars and local_vars['name'] in self.forbidden_mo
                        raise RuntimeError(
                            f"Запрещён импорт модуля: {local_vars['name']}"
                        )

                # Логирование вызова (опционально)
                # print(f"[CALL] {func_name} в {frame.f_code.co_filename}:{frame.f_line

            finally:
                self.is_tracing = False

        elif event == 'line':
            # Логирование строк (можно отключить для производительности)
            # print(f"[LINE] {frame.f_lineno} в {frame.f_code.co_filename}")
            pass

        elif event == 'return':
            self.is_tracing = True

            try:
                self.calls_count -= 1

            finally:
                self.is_tracing = False
```

При поддержке Хабр Курсы РЕКЛАМА

```
            # Обработка исключений
            exc_type, exc_value, exc_traceback = arg
            print(f"[EXCEPTION] {exc_type.__name__}: {exc_value}")
        return self.trace_calls

    def analyze_bytecode(self, code_obj) -> Dict[str, list]:

        """

        Анализ байткода на наличие опасных инструкций (без активной блокировки).

        Полезно для статического анализа перед выполнением.

        """

        warnings = []

        try:
            instructions = dis.get_instructions(code_obj)

            for instr in instructions:
                # Проверка опасных инструкций
                if instr.opname == 'IMPORT_NAME':
                    if instr.argrepr and any(
                        mod in instr.argrepr.lower()
                        for mod in ['os', 'subprocess', 'socket']
                    ):

                        warnings.append({
                            'type': 'dangerous_import',
                            'instruction': instr.opname,
                            'details': f'Импорт: {instr.argrepr}',
                            'line': instr.starts_line
                        })

                elif instr.opname in ('CALL_FUNCTION', 'CALL_FUNCTION_KW'):

                    # В байткоде сложно определить имя вызываемой функции

                    # Можно анализировать предыдущие инструкции LOAD_GLOBAL/LOAD_ATTR
                    pass

                elif instr.opname == 'LOAD_GLOBAL':
                    if instr.argrepr in self.forbidden_functions:
                        warnings.append({
                            'type': 'dangerous_function',
                            'instruction': instr.opname,
                            'details': f'Использование: {instr.argrepr}()',
                            'line': instr.starts_line
                        })

        except Exception as e:
            warnings.append({
                'type': 'analysis_error',
                'details': str(e)
            })

        return {'warnings': warnings, 'is_safe': len(warnings) == 0}
```

При поддержке Хабр Курсы РЕКЛАМА

```python
def safe_executor(code: str, globals_dict: Optional[dict] = None):

    """

    Безопасное выполнение кода в изолированном окружении.

    """

    # Создаём чистое пространство имён
    safe_globals = {

        '__builtins__': {

            'print': print,

            'len': len,

            'range': range,

            'str': str,

            'int': int,

            'float': float,

            'bool': bool,

            'list': list,

            'dict': dict,

            'tuple': tuple,

            'set': set,

            # Исключаем eval, exec, compile, import
        }
    }

    if globals_dict:
        safe_globals.update(globals_dict)

    # Создаём монитор и включаем трассировку
    monitor = ExecutionMonitor()
    old_trace = sys.gettrace()
    sys.settrace(monitor.trace_calls)



    try:
        # Выполняем код
        exec(code, safe_globals)
    except (RuntimeError, RecursionError) as e:
        print(f"[БЕЗОПАСНОСТЬ] Блокировано: {e}")
    except Exception as e:
        print(f"[ОШИБКА] Ошибка выполнения: {e}")
    finally:

        # Восстанавливаем старый трассировщик
```

При поддержке Хабр Курсы РЕКЛАМА

```
class SandboxedFunction:

    """

    Декоратор для безопасного выполнения функций с мониторингом.

    """

    def init(self, max_depth: int = 500):
        self.max_depth = max_depth
        self.monitor = ExecutionMonitor()
        self.monitor.max_depth = max_depth



    def call(self, func):
        @wraps(func)
        def wrapper(*args, **kwargs):
            old_trace = sys.gettrace()
            sys.settrace(self.monitor.trace_calls)

            try:
                return func(*args, **kwargs)

            finally:
                sys.settrace(old_trace)
        return wrapper




…

# Примеры использования

if name == "__main__":
    print("=== Пример 1: Безопасное выполнение ===")
    safe_code = """

print("Это безопасный код")

x = 10 + 20

print(f"Результат: {x}")

"""

    ExecutionMonitor.safe_executor(safe_code)



    print("\n=== Пример 2: Блокировка eval ===")
    dangerous_code = """

print("Попытка использовать eval...")

eval("print('Взлом!')")

"""
```

При поддержке Хабр Курсы РЕКЛАМА

```
    ExecutionMonitor.safe_executor(dangerous_code)



    print("\n=== Пример 3: Статический анализ байткода ===")
    test_code = """

import os
import json



def bad_function():

    eval("print('test')")
    return os.system('ls')

"""

    monitor = ExecutionMonitor()

    # Компилируем код в объект кода

    code_obj = compile(test_code, '<string>', 'exec')

    analysis = monitor.analyze_bytecode(code_obj)
    print(f"Код безопасен: {analysis['is_safe']}")

    for warning in analysis['warnings']:
        print(f"  Предупреждение: {warning['details']}")

    print("\n=== Пример 4: Декоратор для отдельной функции ===")

    @SandboxedFunction(max_depth=100)
    def my_secure_function():
        print("Эта функция защищена")

        # Следующая строка вызовет исключение
        # eval("print('hack')")

    my_secure_function()
```

Объяснить с

Этот код реализует систему мониторинга и ограничения выполнения Python-кода для создания изолированной среды. Его основной целью является ограничить выполнение потенциально опасного кода, запретив опасные функции (`eval` , `exec` , `compile` , `import`), модули (`os` , `subprocess` , `socket` , `sys` , `shutil`) и слишком глубокую рекурсию (защита от переполнения стека).

Мониторинг системных вызовов

Audit hook — самый мощный механизм для production-систем. Он перехватывает все опасные операции на уровне интерпретатора, например импорт модулей, динамическую компиляцию кода, взаимодействие с файловой системой, сетевые запросы и другие потенциально значимые действия. Также он позволяет собирать информацию о внутренних или иначе необнаруживаемых действиях Python или библиотек, написанных на Python.

При поддержке Хабр Курсы РЕКЛАМА

```
import sys
import os

class AuditLogger:

    """Логирование всех системных вызовов"""

    def init(self, blocked_operations=None):
        self.blocked = blocked_operations or [
            'os.system', 'subprocess.Popen', 'open', 'exec', 'import'
        ]
        self.audit_log = []

    def audit_hook(self, event, args):
        """Обработчик аудит-событий"""

        # Логируем событие

        log_entry = {
            'event': event,
            'args': str(args)[:100],  # Ограничиваем длину
            'timestamp': import('time').time()
        }

        self.audit_log.append(log_entry)

        # Проверяем, нужно ли заблокировать
        for blocked in self.blocked:
            if blocked in event:
                print(f" БЛОКИРОВКА: {event} с аргументами {args}")
                raise PermissionError(f"Операция {event} запрещена политикой безопасно
        print(f" {event}: {args}")

    def install(self):
        sys.addaudithook(self.audit_hook)

    def get_report(self):
        return self.audit_log

# Установка аудит-хука

audit = AuditLogger(blocked_operations=['open', 'os.system'])
audit.install()

# Пример 1: Попытка открыть файл

try:
    f = open('/etc/passwd', 'r')
except PermissionError as e:
    print(f"Заблокировано: {e}")

# Пример 2: Попытка выполнить системную команду

try:
    os.system('whoami')
except PermissionError as e:
```

При поддержке Хабр Курсы РЕКЛАМА

```
# Пример 3: Безопасная операция

x = 1 + 2
print(f"Результат: {x}")

# Вывод отчёта

print("\n=== ОТЧЁТ АУДИТА ===")
for entry in audit.get_report():
    print(f"{entry['event']}: {entry['args'][:50]}")
```

Объяснить с

Результат работы данного примера будет следующее:

Для примера 1:

БЛОКИРОВКА: `open` *с аргументами* `('/etc/passwd', 'r')`

Заблокировано: Операция open запрещена политикой безопасности

Для примера 2:

БЛОКИРОВКА: `os.system` *с аргументами* `('whoami',)`

Заблокировано: Операция `os.system` *запрещена политикой безопасности*

=== ОТЧЁТ АУДИТА ===

`open: ('/etc/passwd', 'r')`

`os.system: ('whoami',)`

`builtins.id: (1,)`

Модификация байткода

И наконец, высший пилотаж — изменение кода до его выполнения.

```
import dis
import types

def inspect_bytecode(func):
    """Декомпилируем функцию в байткод"""
    print(f"=== БАЙТКОД {func.__name__} ===")
    dis.dis(func)

    # Получаем сырой байткод
    code = func.__code__
    print(f"\nИнструкции: {code.co_code}")
    print(f"Константы: {code.co_consts}")
    print(f"Имена переменных: {code.co_names}")
    print(f"Локальные переменные: {code.co_varnames}")

def example(a, b):
```

При поддержке Хабр Курсы РЕКЛАМА

```
    return x * 2

inspect_bytecode(example)
```

Объяснить с

Этот код дизассемблирует функцию Python в байткод и показывает её внутреннее устройство. Здесь мы сначала импортируем модули: `dis` (преобразует байткод в человеко-читаемый вид) и `types` (для работы с типами). Далее функция `inspect_bytecode(func)` проводит инспекцию байткода. В результате мы получаем байткод в понятном формате: номера инструкций, сами инструкции и их аргументы.

```
=== БАЙТКОД example ===

  2           0 LOAD_FAST                0 (a)

              2 LOAD_FAST                1 (b)

              4 BINARY_ADD

              6 STORE_FAST               2 (x)



  3           8 LOAD_FAST                2 (x)

             10 LOAD_CONST               1 (2)

             12 BINARY_MULTIPLY

             14 RETURN_VALUE



Инструкции: b'\x97\x00\x97\x01\x17\x00}\x02\x97\x02d\x01\x14\x00S\x00'

Константы: (None, 2)

Имена переменных: ()

Локальные переменные: ('a', 'b', 'x')
```

Объяснить с

В завершении давайте рассмотрим статический анализатор безопасности Python-кода, который исследует байткод функций для обнаружения потенциально опасных паттернов. Мы будем автоматически обнаруживать подозрительный код путём анализа байткода без его выполнения, что позволяет оценить риски до запуска программы.

```
import types
from typing import List, Dict, Any

class SecurityBytecodeInspector:

    """Анализирует байткод на наличие опасных паттернов"""
```

При поддержке Хабр Курсы РЕКЛАМА

```
    DANGEROUS_PATTERNS = {
        'IMPORT_NAME': 'импорт модулей (возможно опасных)',
        'LOAD_GLOBAL': 'использование глобальных функций',
        'CALL_FUNCTION': 'вызов функций',
        'LOAD_ATTR': 'доступ к атрибутам (возможно, вызов методов)',
    }

    HIGHLY_SUSPICIOUS = {
        'eval': 'динамическое выполнение кода',
        'exec': 'динамическое выполнение кода',
        '__import__': 'динамический импорт',
        'open': 'чтение/запись файлов',
        'compile': 'компиляция кода',
        'globals': 'доступ к глобальным переменным',
        'locals': 'доступ к локальным переменным',
        '__builtins__': 'доступ к встроенным функциям',
    }


    def init(self):
        self.findings = []

    def analyze_function(self, func) -> Dict[str, Any]:
        """Анализ функции на безопасность"""

        self.findings = []

        print(f"\n АНАЛИЗ БЕЗОПАСНОСТИ: {func.__name__}")
        print("="*50)

        # Получаем байткод
        code = func.__code__

        # Анализируем константы
        self._check_constants(code.co_consts)

        # Анализируем имена
        self._check_names(code.co_names)


        # Детальный разбор инструкций
        self._analyze_instructions(func)

        return {
            'function': func.__name__,
            'is_safe': len(self.findings) == 0,
            'findings': self.findings,
            'constants': code.co_consts,
            'global_names': code.co_names,
            'local_names': code.co_varnames
        }



    def checkconstants(self, constants):
        """Проверка констант на опасные строки"""
        for const in constants:
            if isinstance(const, str):
                # Проверка на подозрительные строки
```

При поддержке Хабр Курсы РЕКЛАМА

```
                for sus in suspicious:
                    if sus in const:
                        self.findings.append({
                            'severity': 'HIGH',
                            'type': 'SUSPICIOUS_STRING',
                            'detail': f'Найдена подозрительная строка: "{const}"',
                            'suggestion': 'Возможно, код пытается выполнить динамически

                        })



    def checknames(self, names):
        """Проверка используемых имён"""
        for name in names:
            if name in self.HIGHLY_SUSPICIOUS:
                self.findings.append({
                    'severity': 'CRITICAL',
                    'type': 'DANGEROUS_FUNCTION',
                    'detail': f'Обнаружена опасная функция: {name}()',
                    'suggestion': self.HIGHLY_SUSPICIOUS[name]
                })

    def analyzeinstructions(self, func):
        """Детальный анализ инструкций байткода"""
        print("\n📋 ПОИСК ОПАСНЫХ ИНСТРУКЦИЙ:")


        for instr in dis.get_instructions(func):

            # Проверяем инструкции импорта

            if instr.opname == 'IMPORT_NAME':
                self.findings.append({
                    'severity': 'MEDIUM',
                    'type': 'IMPORT',
                    'detail': f'Импорт модуля: {instr.argrepr}',
                    'line': instr.starts_line
                })
                print(f"  Найден импорт: {instr.argrepr} (строка {instr.starts_line})")



            # Проверяем загрузку глобальных переменных
            elif instr.opname == 'LOAD_GLOBAL' and instr.argrepr in self.HIGHLY_SUSPICI
                self.findings.append({
                    'severity': 'HIGH',
                    'type': 'SUSPICIOUS_GLOBAL',
                    'detail': f'Загрузка опасной глобальной функции: {instr.argrepr}',
                    'line': instr.starts_line
                })
                print(f"  КРИТИЧНО: {instr.argrepr}() (строка {instr.starts_line})")

            # Проверка доступа к атрибутам (возможно, методы объектов)
            elif instr.opname == 'LOAD_ATTR':
                self.findings.append({
                    'severity': 'LOW',
                    'type': 'ATTRIBUTE_ACCESS',
```

При поддержке Хабр Курсы РЕКЛАМА

```
                    'line': instr.starts_line
                })

    def generate_report(self) -> str:
        """Генерация отчёта о безопасности"""
        if not self.findings:
            return " Код безопасен для выполнения"

        report = [" ОБНАРУЖЕНЫ УЯЗВИМОСТИ:\n"]

        critical = [f for f in self.findings if f['severity'] == 'CRITICAL']

        high = [f for f in self.findings if f['severity'] == 'HIGH']
        medium = [f for f in self.findings if f['severity'] == 'MEDIUM']

        if critical:
            report.append(f" КРИТИЧЕСКИЕ ({len(critical)}):")
            for finding in critical:
                report.append(f"   - {finding['detail']}")
                report.append(f"     → {finding['suggestion']}")

        if high:

            report.append(f"\n ВЫСОКИЙ РИСК ({len(high)}):")
            for finding in high:
                report.append(f"   - {finding['detail']}")

        if medium:

            report.append(f"\n СРЕДНИЙ РИСК ({len(medium)}):")

            for finding in medium:
                report.append(f"   - {finding['detail']}")

        return "\n".join(report)


# ===== ПРИМЕРЫ ИСПОЛЬЗОВАНИЯ =====



def safe_calculation(x, y):

    """Безопасная функция - только математика"""
    return (x + y) * 2



def dangerous_file_reader():

    """Опасная функция - читает файлы"""
    f = open('/etc/passwd', 'r')
    content = f.read()
    f.close()
    return content



def malicious_code_executor():
```

При поддержке Хабр Курсы РЕКЛАМА

```
    """Зловредный код - динамическое выполнение"""
    user_input = "__import__('os').system('rm -rf /')"
    eval(user_input)



def stealthy_attacker():

    """Скрытый зловред - через константы"""
    # Прячем опасный код в константе
    dangerous_code = "eval('print(\"HACKED\")')"
    # Позже может быть выполнен
    return dangerous_code



# Запуск анализа

inspector = SecurityBytecodeInspector()



# 1. Анализ безопасной функции
print(" ТЕСТ 1: Безопасная функция")
result = inspector.analyze_function(safe_calculation)
print(inspector.generate_report())



# 2. Анализ опасной функции

print("\n" + "="*60)
print(" ТЕСТ 2: Опасная функция (чтение файлов)")
result = inspector.analyze_function(dangerous_file_reader)
print(inspector.generate_report())



# 3. Анализ вредоносной функции

print("\n" + "="*60)
print(" ТЕСТ 3: Вредоносная функция (eval)")
result = inspector.analyze_function(malicious_code_executor)
print(inspector.generate_report())



# 4. Анализ скрытой угрозы

print("\n" + "="*60)
print(" ТЕСТ 4: Скрытая угроза (в константах)")
result = inspector.analyze_function(stealthy_attacker)
print(inspector.generate_report())
```

Объяснить с

При поддержке Хабр Курсы РЕКЛАМА

Опасный код с `open()` вернет следующее:

> *КРИТИЧЕСКИЕ (1):*
>
> *— Обнаружена опасная функция: `open()`*
>
> *→ чтение/запись файлов*
>
> *Вредоносный код с `eval()`*
>
> *КРИТИЧЕСКИЕ (2):*
>
> *— Обнаружена опасная функция: `eval()`*
>
> *→ динамическое выполнение кода*
>
> *— Найдена подозрительная строка: `«__import__('os').system('rm -rf /')»`*
>
> *В примере со скрытой угрозой:*
>
> *ВЫСОКИЙ РИСК (1):*
>
> *— Найдена подозрительная строка: `«eval('print(\»HACKED\«)')»`*

Заключение

Мы рассмотрели четыре уровня контроля над выполнением Python-кода: статический анализ, `sys.settrace()` — для построчного мониторинга, `sys.addaudithook()` — для перехвата системных операций и анализ байткода — для внедрения контрактов и проверок

Здесь стоит отметить, что ни один из методов не даёт 100% гарантии в чистом Python.
Для реальных `high-security` проектов комбинируйте эти техники с изоляцией на уровне ОС (контейнеры, seccomp, AppArmor).

Python удобен не потому, что «простой», а потому что позволяет быстро добраться до сложных вещей: рантайма, байткода, AST, системных хуков и архитектурных решений вокруг безопасности. Но чтобы уверенно работать с такими механизмами в продакшене, уже недостаточно знать синтаксис и популярные библиотеки.

На курсе **«Python-разработчик. Продвинутый уровень»** разбираем Python глубже:
от внутреннего устройства языка и асинхронности до проектирования надежных backend-сервисов, работы с производительностью и инженерных практик, которые нужны разработчику уровня middle+.

> 📌 *Перед стартом можно пройти бесплатное тестирование: оно поможет оценить текущий уровень, понять, насколько программа подходит под ваши задачи, и увидеть, какие темы стоит подтянуть до обучения.* → **`[Оценить свой уровень]`**

При поддержке Хабр Курсы РЕКЛАМА

sys.settrace, runtime-контроль

Хабы: Блог компании OTUS, Python, Информационная безопасность, Программирование

0 4 0

Редакторский дайджест

Присылаем лучшие статьи раз в месяц

Электропочта

Подписаться

Оставляя почту, я принимаю Политику конфиденциальности и даю согласие на получение рассылок

512K+
Охват за 30 дней

OTUS

Сайт ВКонтакте Telegram

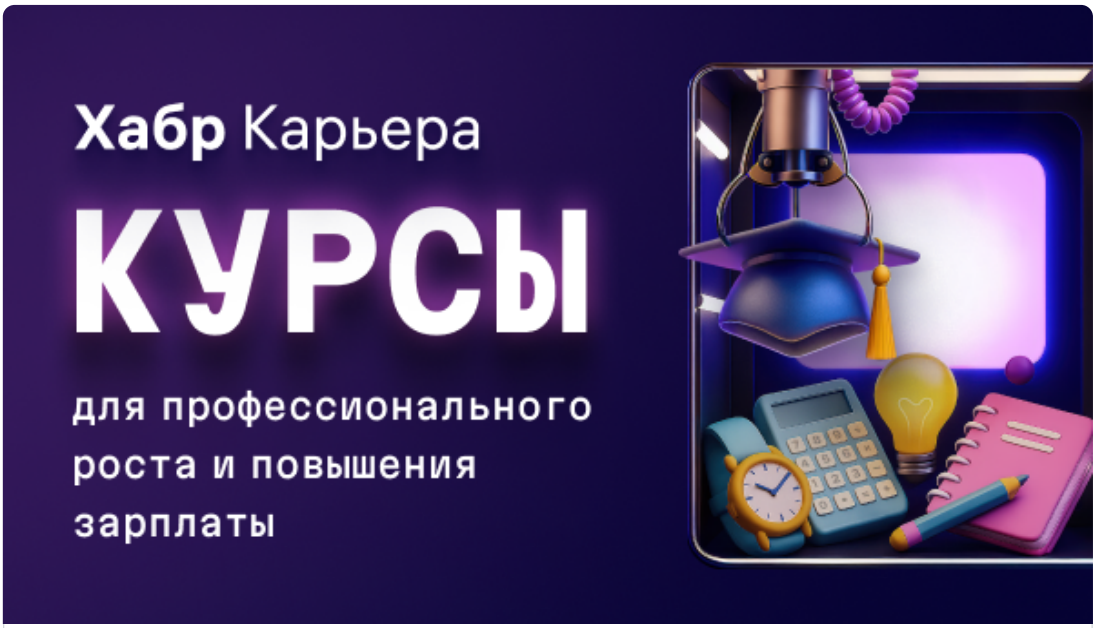
64K+ Охват за 30 дней
36 Карма

@Andrey_Biryukov

Пользователь

Подписаться

Поток Бэкенд доступен 24/7 благодаря поддержке друзей Хабра



Хабр Курсы для бэкендеров РЕКЛАМА

Практикум, Хекслет, SkyPro, авторские курсы — собрали всех и попросили скидки. Осталось выбрать!

Перейти

При поддержке Хабр Курсы РЕКЛАМА

Комментарии

Здесь пока нет ни одного комментария, вы можете стать первым!

Зарегистрируйтесь на Хабре, чтобы оставить комментарий

Публикации

ЛУЧШИЕ ЗА СУТКИ ПОХОЖИЕ

samako
8 часов назад

Давайте уже сделаем отдельный хаб: «Я собрал приложение за вечер — зачем теперь разработчики?»

Простой 2 мин 9.7K

Мнение

+97 9 45

inkedsymon
11 часов назад

Муравьи против трансформеров: старый алгоритм 1992 года, который вернулся

Средний 9 мин 8.7K

Обзор

+55 35 1

da1ight
5 часов назад

Голосуй, или проиграешь! разбор фишинга, который пытается угнать Telegram

Средний 13 мин 7.8K

Кейс

+30 13 16

strannik96
10 часов назад

Чебурнет близко

Простой 5 мин 9K

Обзор

+29 22 56

techno_mot
7 часов назад

При поддержке Хабр Курсы РЕКЛАМА

дайджест

8 мин 6.7K

Дайджест

+28 5 2

raipc
10 часов назад

Как Monium приручил GC: разбираемся со сборщиками мусора в observability-платформе

12 мин 7.1K

+25 6 0

programmerguru
12 часов назад

Как запускать LLM-агентов без облачных API: VPS, локальные модели и требования к железу

Средний 6 мин 10K

Туториал

+23 45 4

PatientZero
13 часов назад

Баги, которые не ловит Rust

Средний 12 мин 9K

Перевод

+23 22 0

morett1m
12 часов назад

Что делает match после того, как вы нажали Compile

Средний 11 мин 7.6K

Обзор

+22 17 1

AnSa8
5 часов назад

OneOCR — скрытая OCR внутри Windows 11

Простой 7 мин 6.2K

Туториал Из песочницы

+21 8 2

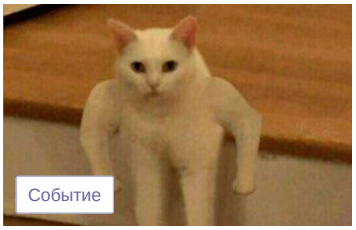
Как мы пять лет закрывали техдолг, а потом настроили LLM и ускорились вдвое

При поддержке Хабр Курсы РЕКЛАМА

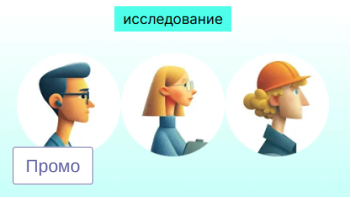
Турбо

Показать еще

МИНУТОЧКУ ВНИМАНИЯ



Качаем к лету хард- и софт-скиллы на айтишных ивентах



Лучшие практики по работе с молодыми специалистами

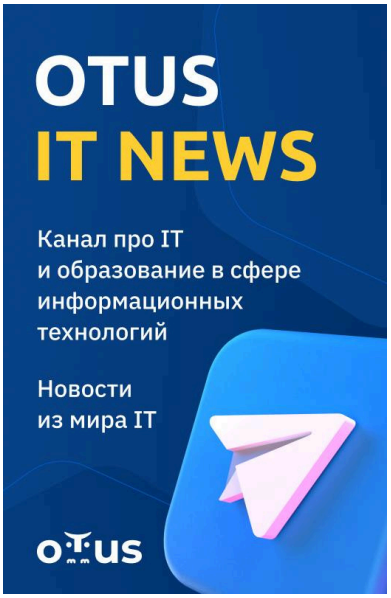
Удалёнка в фарме: большие данные и никакой магии

ИНФОРМАЦИЯ

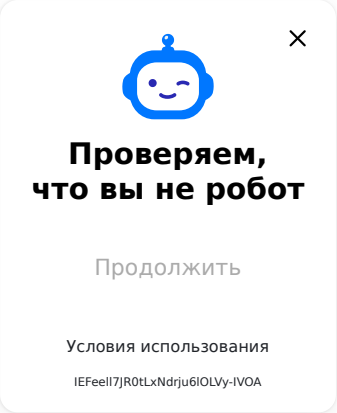
Сайт	otus.ru
Дата регистрации	22 марта 2017
Дата основания	1 апреля 2017
Численность	101–200 человек
Местоположение	Россия
Представитель	OTUS

При поддержке Хабр Курсы РЕКЛАМА

ВИДЖЕТ



ВКОНТАКТЕ



БЛОГ НА ХАБРЕ

1 час назад

Сделаем Python безопасным... снова

3.1K 0

5 часов назад

Почему чат-бот не справляется с юридической работой и чем от него отличается ИИ-агент

4.6K 1

9 часов назад

Ваш Kafka lag врёт: как настроить алерты по реальной задержке, а не по числу сообщений

5.4K 0

12 часов назад

MCP в Cursor IDE: подключаем AI-агенту внешние данные без возни с API

6.5K 1

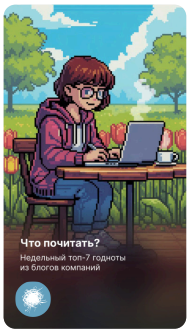
При поддержке Хабр Курсы РЕКЛАМА

вчера в 00:05

Почему CRM в Битрикс24 тормозит на 50К сделок и что с этим делать

12K 4

ИСТОРИИ

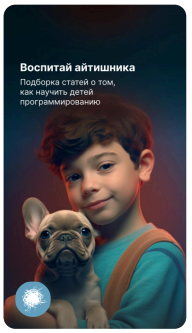


Лучшее из блогов компаний

Остановка «Ретейл будущего»

Там на неведомых дорожках...

Схватил за мозг



Как учить детей программированию

© 2006–2026, Habr